<!--Estudio Shonos-->

Subida de un punto

<Por="Pablo Belascoain,Agustín Borreguero Castro,Adán Gavira Palacios
"/>

Contenidos

01 Introducción

02 Objetivo

03 Arquitectura

04 Funcionalidades

05 Persistencia

06 Implementaciones

07 Pruebas

Introducción {

Este proyecto consiste en el desarrollo de una misma aplicación gráfica implementada en tres tecnologías distintas:

- Java Swing
- JavaFX
- Android

El objetivo principal ha sido analizar y comparar diferentes enfoques de desarrollo de interfaces gráficas, manteniendo una lógica funcional común y aplicando criterios profesionales de diseño GUI, estructura del programa y calidad técnica.

- ▪ Implementación multiplataforma
- ▪ Separación modelo - vista - lógica
- ▪ Aplicación de principios GUI
- ▪ Persistencia local de datos
- ▪ Pruebas unitarias
- ▪ Sistema de ayuda integrado
- ▪ Distribución ejecutable

}

Arquitectura del Proyecto {

Logica común

- Modelo independiente de la interfaz
- Gestión de datos centralizada
- Validaciones separadas
- Persistencia desacoplada

Separación por capas

- Modelo
- Vista
- Controlador / Lógica
- Utilidades

El proyecto se ha diseñado siguiendo una arquitectura modular que permite reutilizar la lógica en las tres tecnologías.
La interfaz cambia según el entorno (Swing, JavaFX o Android), pero el modelo y la lógica funcional se mantienen coherentes.

}

DIFERENCIAS ENTRE TECNOLOGÍAS {

01

Swing: interfaz construida por código

02

JavaFX: interfaz declarativa con FXML

03

Android: layouts XML + ciclo de vida

04

Swing: eventos manuales

05

JavaFX: controladores vinculados

06

Android: Activities y Fragments

07

JavaFX y Android permiten mejor separación visual

08

Android añade gestión de ciclo de vida y adaptación a pantalla

}

Retos del proyecto {

Dificultades encontradas

- Durante el desarrollo del proyecto, uno de los principales retos fue mantener una coherencia funcional entre tres tecnologías con enfoques de interfaz completamente distintos.
- Cada entorno (Swing, JavaFX y Android) gestiona los eventos, la navegación y la estructura de la interfaz de manera diferente, lo que obligó a adaptar la implementación sin alterar la lógica común.
- Además, fue necesario aplicar correctamente la separación entre modelo y vista, evitar duplicar lógica en la interfaz y garantizar que las tres versiones cumplieran exactamente los mismos requisitos funcionales.

Soluciones aplicadas

- Para resolver estas dificultades, se diseñó previamente un modelo común independiente de la interfaz, permitiendo reutilizar la lógica en las tres versiones.
- Se estructuraron los proyectos en paquetes diferenciados (modelo, vista, controlador y utilidades), lo que facilitó el mantenimiento y las pruebas unitarias.
- Finalmente, se aplicó una revisión sistemática mediante checklist de requisitos obligatorios para asegurar que ninguna versión quedara incompleta antes de la entrega y defensa.

}

Pruebas Unitarias {

Para garantizar la calidad técnica del proyecto, se han implementado pruebas unitarias centradas en la lógica de negocio común a las tres versiones.
Se han desarrollado un mínimo de 15 pruebas utilizando JUnit, enfocadas en:

- Validaciones de campos obligatorios
- Búsqueda por texto
- Filtrado por categoría
- Persistencia de datos
- Gestión de errores

Las pruebas se aplican únicamente sobre la lógica del modelo, evitando incluir código de interfaz no testeable.

}

Principios GUI aplicados {

Diseño y coherencia visual

- Aplicación de jerarquía visual clara (títulos, acciones primarias y secundarias).
- Coherencia entre pantallas en las tres versiones.
- Uso consistente de tipografía, colores y espaciado.
- Alineación uniforme de botones y campos de entrada.
- Diseño adaptado a cada entorno respetando sus convenciones.

Experiencia de usuario (UX)

- Confirmaciones antes de eliminar elementos.
- Feedback visual inmediato ante errores de validación.
- Mensajes claros y comprensibles para el usuario.
- Navegación intuitiva entre pantallas.
- Sistema de ayuda accesible y contextual.

}

DISTRIBUCIÓN DEL PROYECTO

{

El proyecto ha sido preparado para su distribución real en las tres tecnologías implementadas, garantizando su ejecución independiente del entorno de desarrollo.
Cada versión ha sido compilada y exportada como aplicación ejecutable, verificando su correcto funcionamiento fuera del IDE.
Se ha comprobado la instalación, apertura, navegación y cierre correcto en todos los casos, asegurando que la aplicación está lista para su uso final.

- ▪ Swing → Generación de archivo JAR ejecutable
- ▪ JavaFX → Generación de ejecutable independiente
- ▪ Android → Generación de APK firmado (release)
- ▪ Verificación de ejecución fuera del IDE
- ▪ Evidencias de instalación y funcionamiento
- ▪ Comprobación de cierre correcto

}

Android Studio {

◆ Implementación en Android

La versión Android ha sido desarrollada en Android Studio utilizando Activities y/o Fragments, con layouts definidos en XML y correcta gestión de recursos (strings, colores y estilos).

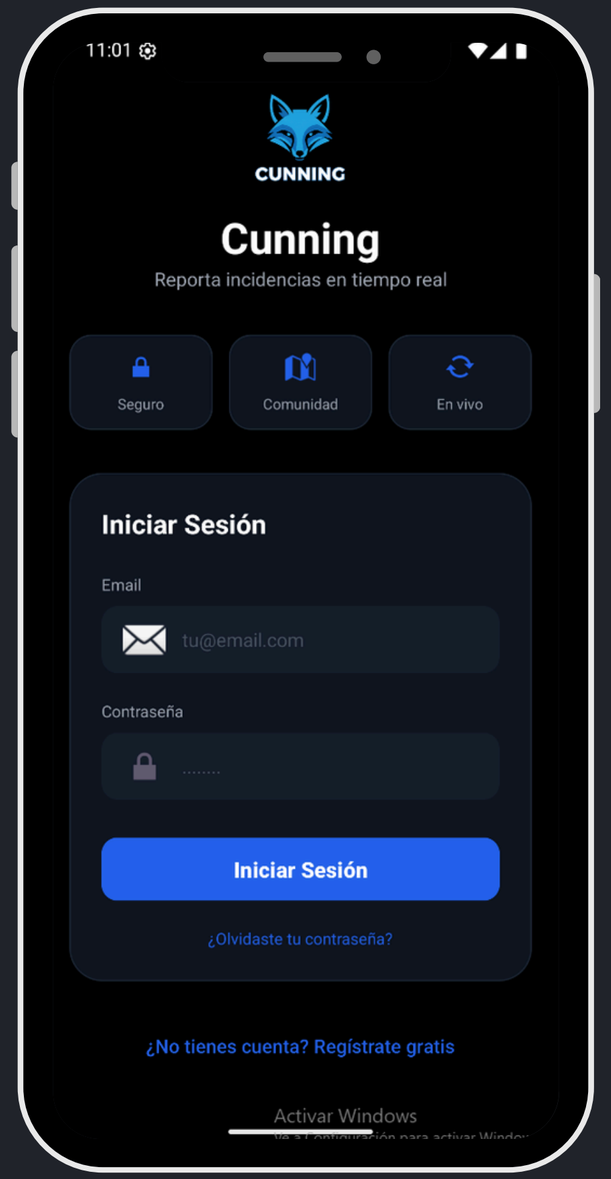
Se ha aplicado una estructura modular separando modelo, lógica y presentación, evitando incluir lógica crítica dentro de la interfaz gráfica.

◆ Requisitos cumplidos

- Pantalla principal con listado (mínimo 15 elementos)
  - Vista de detalle completa
  - Alta, modificación y eliminación con validaciones
  - Búsqueda por texto y filtrado por categoría
  - Persistencia local de datos
  - Sistema de ayuda accesible y contextual
  - Adaptación básica a distintos tamaños de pantalla
  - Generación de APK firmado para distribución

}

Android Studio {



}

JavaFX {

◆ Implementación Técnica

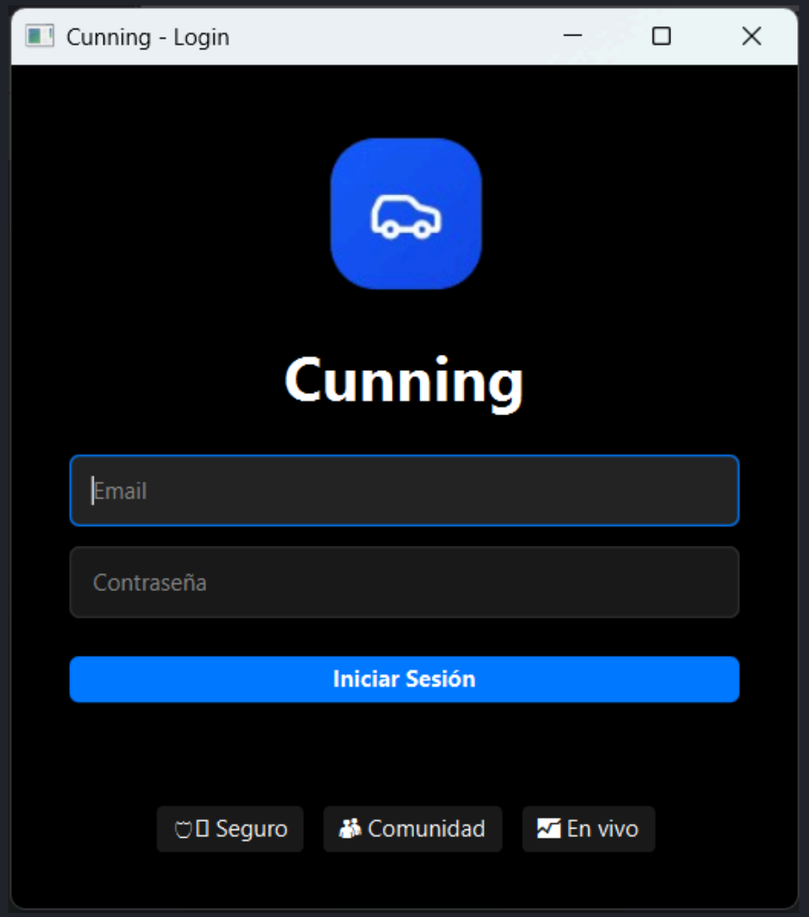
La aplicación de escritorio se ha desarrollado con JavaFX 21 e IntelliJ IDEA, utilizando archivos FXML para la interfaz y CSS para el diseño visual. Se emplea una arquitectura MVC con un SceneManager para gestionar la navegación entre ventanas de forma eficiente.

◆ Requisitos cumplidos

- **Interfaz Dinámica:** Pantalla principal con listado de comunidades mediante tarjetas interactivas.
- **Diseño Visual:** Tema oscuro coherente gestionado íntegramente mediante style.css.
- **Lógica de Control:** Vinculación de eventos a través de HelloController para altas e interacciones.
- **Navegación:** Sistema centralizado de intercambio de escenas entre Login y Dashboard.
- **Gestión de Recursos:** Estructura de Maven optimizada para la carga de imágenes, FXML y estilos.
- **Estabilidad:** Control de excepciones en la carga de recursos para evitar fallos de ejecución.

}

JavaFX {


Cunning - Login
Cunning
Email
Contraseña
Iniciar Sesión
Seguro
Comunidad
En vivo

Cunning - Dashboard
Usuario: adan
Bienvenido de nuevo
+ Mis comunidades
Madrid Centro
Incidencias en el centro de Madrid y M-30
1247 Usuarios
8 activas
Interactuar
A-1 Norte
Autopista A-1 dirección norte
856 Usuarios
3 activas
Interactuar
Cerrar Sesión

}

Java Swing {

- Implementación Java Swing

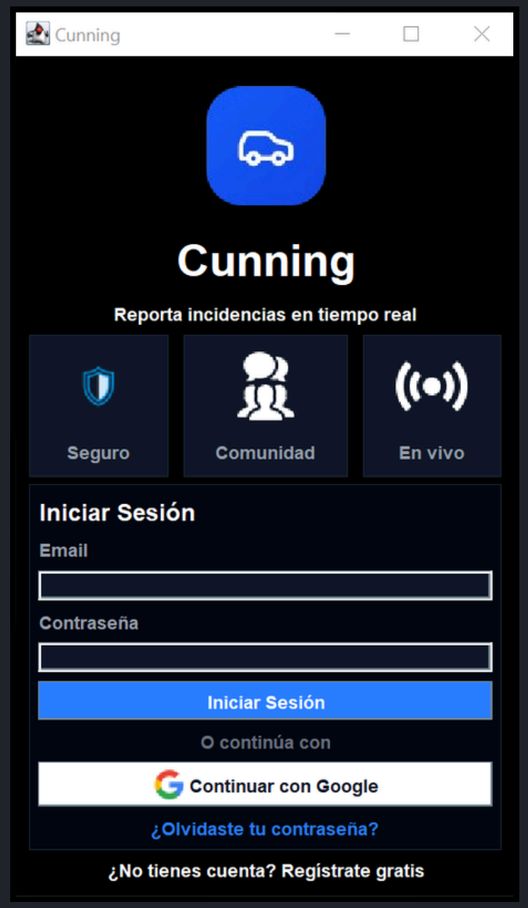
La aplicación de escritorio se ha desarrollado utilizando Java Swing en IntelliJ IDEA. La interfaz gráfica se ha creado mediante GUI Designer, utilizando archivos .form y clases Java para gestionar los componentes visuales. La navegación entre pantallas se gestiona mediante JFrame, JPanel y JDialog, permitiendo pasar de la pantalla de inicio de sesión a la pantalla principal de la aplicación.

- Requisitos cumplidos

- **Interfaz gráfica: Pantalla de inicio de sesión y pantalla principal.**
- **Gestión de ventanas: Navegación entre pantallas mediante JFrame.**
- **Control de eventos: Uso de ActionListener para gestionar las acciones del usuario.**
- **Confirmación de acciones: Ventanas de diálogo con JOptionPane.**
- **Diseño visual: Interfaz creada con GUI Designer de IntelliJ.**
- **Distribución: Generación de un archivo ejecutable .jar para ejecutar la aplicación.**

}

Java Swing {


Cunning
Cunning
Reporta incidencias en tiempo real
Seguro
Comunidad
En vivo
Iniciar Sesión
Email
Contraseña
Iniciar Sesión
O continúa con
Continuar con Google
¿Olvidaste tu contraseña?
¿No tienes cuenta? Regístrate gratis

Cunning
Hola, Usuario
Bienvenido de nuevo
Mis comunidades
Madrid Centro
Incidencias en el centro de Madrid y M-30
1247 Usuarios
8 activas
Interactuar
A-1 Norte
Autopista A-1 dirección norte
856 Usuarios
3 activas
Interactuar
Cerrar Sesión

}

CONCLUSIONES {

El desarrollo del proyecto en tres tecnologías distintas ha permitido analizar en profundidad diferentes enfoques de construcción de interfaces gráficas, manteniendo una lógica funcional común.

Se ha demostrado la importancia de la separación entre modelo e interfaz, la aplicación de principios GUI y la implementación de pruebas unitarias para garantizar calidad técnica.

Cada tecnología presenta ventajas y limitaciones:

- Swing ofrece simplicidad estructural, pero mayor carga manual en la interfaz.
- JavaFX mejora la separación visual mediante FXML y CSS.
- Android introduce mayor complejidad debido al ciclo de vida y arquitectura móvil, pero permite una estructura más escalable.

El proyecto cumple los requisitos funcionales, estructurales y de calidad exigidos, demostrando comprensión técnica real de cada entorno.

}

<!--Estudio Shonos-->

Gracias {

<Por="Pablo Belascoain, Agustín Borreguero Castro y Adán Gavira Palacios "/>